

Contents

1	Introduction	3
1.1	Background and motivation	3
1.2	Problem statement	3
2	Aim, Objectives, and Scope	3
2.1	Aim	3
2.2	Objectives	3
2.3	Project scope	3
3	Requirements Analysis	4
3.1	Functional requirements	4
3.2	Non-functional requirements	5
4	System Design	6
4.1	Conceptual architecture	6
4.2	Major software components	6
4.3	Processing flow	6
5	Knowledge-Base Design	7
5.1	Knowledge-base inventory	7
6	Inference and Ranking Method	9
6.1	Set-based matching	9
6.2	Worked example	9
6.3	Algorithm	10
7	Graphical User Interface	10
7.1	Window and visual organization	10
7.2	Scrollable symptom checklist	10
7.3	Result presentation and widget state	11

8	Implementation Discussion	11
8.1	Event-driven execution	11
8.2	Input validation	12
8.3	Separation of concerns	12
9	Testing and Evaluation	12
9.1	Illustrative expected outputs	13
9.2	Evaluation against objectives	13
10	Limitations, Ethics, and Safety	14
10.1	Technical and reasoning limitations	14
10.2	Ethical use	14
10.3	Emergency considerations	15
11	Future Enhancements	15
12	Conclusion	15
A	Data Dictionary	16
B	User Guide	16

1 Introduction

An expert system is a computer program that applies stored domain knowledge and a reasoning procedure to a user's facts. In this project, the facts are symptoms selected by the user, while the domain knowledge is a collection of possible medical conditions and their associated symptoms. The program compares these two sets and reports the closest matches.

The application is intended to illustrate how a compact, understandable inference mechanism can be connected to a usable desktop interface. It does not attempt to reproduce the reasoning of a qualified clinician. Real diagnosis depends on information absent from this program, including symptom duration and severity, medical history, examination findings, laboratory results, medication use, age, and risk factors. Accordingly, both the interface and this report treat its output as educational guidance only.

1.1 Background and motivation

People often find it difficult to understand how several symptoms might be related. A rule-based demonstration is useful because its knowledge is explicit: each result can be traced directly to a stored list of symptoms. In contrast to an opaque predictive model, this program can state exactly which selected symptoms produced each match. This transparency makes the project suitable for teaching knowledge representation, set operations, ranking algorithms, and event-driven GUI development.

1.2 Problem statement

The project addresses the following software problem: design a desktop program that allows a user to select symptoms, compares those symptoms with a predefined medical knowledge base, ranks the related conditions, and presents understandable results and safety-conscious recommendations. The program must also validate empty input and allow the interface to be reset easily.

2 Aim, Objectives, and Scope

2.1 Aim

The aim is to develop an educational rule-based expert system that demonstrates symptom matching through a simple and accessible Tkinter interface.

2.2 Objectives

2.3 Project scope

The scope includes a local, single-user desktop application; a fixed knowledge base of 24 conditions; binary symptom selection; deterministic matching; presentation of the top five results; general

Table 1: Project objectives and evidence of implementation

ID	Objective	Implementation evidence
O1	Represent medical knowledge in a form that is easy to inspect and extend.	A list of dictionaries stores the name, symptoms, and advice for every condition.
O2	Collect user symptoms without requiring typed medical terms.	Checkboxes are generated from a sorted set of all unique symptoms.
O3	Compare the user's selection with every condition.	The <code>diagnose()</code> function uses set intersection to find matching symptoms.
O4	Rank possible conditions consistently.	A normalized score is calculated and results are sorted in descending order.
O5	Explain why each condition appears.	The result area lists the score, matched symptoms, and relevant advice.
O6	Provide a straightforward desktop interface.	Tkinter frames, labels, buttons, a scrollbar, and a text widget organize the workflow.
O7	Communicate the system's safety limitation.	A prominent notice states that the application does not provide a medical diagnosis.

advice; input validation; and a clear/reset operation. The scope excludes user accounts, persistent records, internet access, language processing, clinician verification, probabilistic learning, medicine dosage, emergency triage, and integration with electronic health records.

3 Requirements Analysis

3.1 Functional requirements

Table 2: Functional requirements

ID	Requirement	Expected behavior
FR1	Display available symptoms	The interface presents every unique symptom as a labeled checkbox.
FR2	Support multiple selections	The user can select any combination of symptoms before requesting an assessment.
FR3	Validate input	Selecting <i>Diagnose</i> with no checked symptom opens a warning dialog and does not run the inference engine.
FR4	Generate matches	Each selected symptom is compared with every condition in the knowledge base.

ID	Requirement	Expected behavior
FR5	Calculate relevance	The system divides the number of matching symptoms by the total number stored for that condition.
FR6	Rank output	Matching conditions are ordered from the highest score to the lowest score.
FR7	Limit output length	No more than the first five ranked conditions are displayed.
FR8	Explain results	Each entry includes the condition, percentage, matching symptoms, and advice.
FR9	Clear the interface	The <i>Clear</i> button unchecks all symptoms and removes existing output.
FR10	Display a disclaimer	Every completed assessment ends with a warning that the tool is not a medical diagnosis.

3.2 Non-functional requirements

Table 3: Non-functional requirements

Quality	Requirement	Design response
Usability	A first-time user should understand the main workflow without training.	Descriptive group labels, familiar checkboxes, and clearly colored action buttons are used.
Responsiveness	Results should appear immediately for the supplied knowledge-base size.	Matching is performed in memory with small Python collections.
Maintainability	Conditions and advice should be editable without changing GUI logic.	Domain data are separated from the interface class.
Portability	The application should run on major desktop platforms with Python.	Tkinter is part of the standard Python distribution on many installations.
Explainability	A user should be able to see the evidence behind a result.	The application exposes the intersection of selected and stored symptoms.
Privacy	The demonstration should not transmit or store entered symptoms.	All processing occurs in memory, with no database, files, or network connection.
Safety	Output must not be presented as confirmed diagnosis.	Advice is general, and a medical disclaimer accompanies the results.

4 System Design

4.1 Conceptual architecture

The program follows a simple three-part architecture. The presentation layer collects symptoms and displays output. The inference engine converts the selection into scores and rankings. The knowledge base supplies the conditions, symptom lists, and advice. Although all parts reside in one Python file, their responsibilities are logically separated.

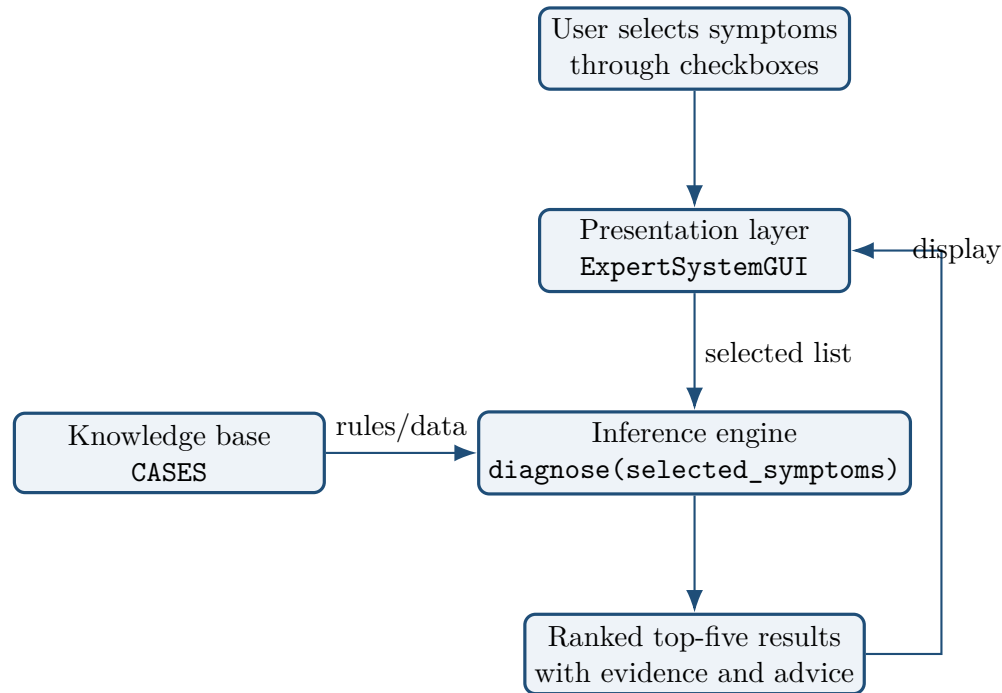


Figure 1: Conceptual architecture of the application

4.2 Major software components

4.3 Processing flow

The sequence begins when Python creates the root window and initializes `ExpertSystemGUI`. The constructor calls four interface-building methods. Each symptom receives a `BooleanVar`, which records whether its checkbox is selected. When the user presses *Diagnose*, the program builds a list containing only the symptoms whose variables are true. Empty input produces a warning. Valid input is sent to `diagnose()`, and the returned list is formatted in the result widget. The widget is normally disabled so that the output behaves as read-only text.

Table 4: Software components and responsibilities

Component	Primary data	Responsibility
CASES	List of dictionaries	Stores all condition names, known symptoms, and advice.
ALL_SYMPTOMS	Sorted list of strings	Removes duplicate symptoms and provides a predictable order for the GUI.
diagnose()	Selected symptom list	Computes intersections, scores matching cases, and returns a ranked list.
ExpertSystemGUI	Widget state and Boolean variables	Builds the window, handles events, and formats results.
run_diagnosis()	Checked symptoms and ranked results	Validates input, calls the engine, and populates the result widget.
clear_selection()	Checkbox variables and result text	Restores the interface to its initial state.
Main block	Tk root window	Creates the application and starts Tkinter's event loop.

5 Knowledge-Base Design

Each knowledge-base record has three fields. The **name** field is the label shown in the results. The **symptoms** field is a list of normalized lowercase phrases used during matching. The **advice** field is general guidance presented alongside a match. This uniform schema allows the inference engine to process every condition with the same logic.

Listing 1: General structure of one knowledge-base record

```

1 {
2   "name": "Condition name",
3   "symptoms": ["symptom one", "symptom two"],
4   "advice": "General recommendation shown to the user."
5 }
```

5.1 Knowledge-base inventory

Table 5: Conditions and symptoms represented in the knowledge base

No.	Condition	Associated symptoms	Count
1	Common Cold	Runny nose, sneezing, sore throat, cough	4
2	Influenza (Flu)	Fever, headache, body aches, cough, fatigue	5

No.	Condition	Associated symptoms	Count
3	COVID-19	Fever, cough, fatigue, loss of taste, loss of smell	5
4	Allergic Rhinitis	Sneezing, runny nose, itchy eyes, watery eyes	4
5	Migraine	Severe headache, nausea, sensitivity to light, dizziness	4
6	Sinusitis	Facial pain, blocked nose, headache, runny nose	4
7	Asthma	Shortness of breath, wheezing, chest tightness, cough	4
8	Bronchitis	Cough, chest discomfort, fatigue, shortness of breath	4
9	Pneumonia	Fever, cough, shortness of breath, chest pain, fatigue	5
10	Gastroenteritis	Nausea, vomiting, diarrhea, stomach pain	4
11	Food Poisoning	Vomiting, diarrhea, stomach pain, fever	4
12	Acid Reflux	Heartburn, chest discomfort, sour taste, belching	4
13	Dehydration	Thirst, dry mouth, dizziness, fatigue	4
14	Anemia	Fatigue, dizziness, pale skin, shortness of breath	4
15	Urinary Tract Infection	Painful urination, frequent urination, lower abdominal pain, fever	4
16	Tension Headache	Headache, neck pain, stress, fatigue	4
17	Insomnia	Difficulty sleeping, daytime fatigue, difficulty concentrating, stress	4
18	Conjunctivitis	Red eyes, itchy eyes, watery eyes, eye discharge	4
19	Dermatitis	Skin rash, itchy skin, red skin, dry skin	4
20	Chickenpox	Fever, skin rash, itchy skin, fatigue	4
21	Dengue Fever	High fever, severe headache, joint pain, skin rash	4
22	Malaria	Fever, chills, sweating, headache, fatigue	5
23	Strep Throat	Sore throat, fever, swollen glands, difficulty swallowing	4
24	Ear Infection	Ear pain, fever, hearing difficulty, fatigue	4
Total symptom associations			101

The system derives `ALL_SYMPTOMS` rather than maintaining a second manual symptom list. A nested comprehension visits every case and every symptom. Converting the values to a set removes duplicates, while `sorted()` produces alphabetical display order. This approach prevents the checklist from drifting out of synchronization with the knowledge base when a new symptom is added.

Listing 2: Automatic construction of the symptom checklist

```

1 ALL_SYMPTOMS = sorted(
2     set(
3         symptom
4         for case in CASES
5         for symptom in case["symptoms"]
6     )
7 )

```

6 Inference and Ranking Method

6.1 Set-based matching

Let U be the universe of symptoms in the checklist, $S \subseteq U$ be the set selected by the user, and $C_i \subseteq U$ be the symptom set stored for condition i . The evidence supporting condition i is the intersection

$$M_i = S \cap C_i. \quad (1)$$

A condition is included only when $|M_i| > 0$. The program then calculates

$$\text{score}(i) = \frac{|S \cap C_i|}{|C_i|}, \quad \text{percentage}(i) = 100 \times \text{score}(i). \quad (2)$$

The denominator is the number of symptoms stored for the condition, not the number selected by the user. Consequently, the percentage expresses *coverage of the condition's stored symptom profile*. It is not a probability that the user has the condition, and the values across conditions do not sum to 100%.

6.2 Worked example

Suppose the user selects

$$S = \{\text{fever, cough, fatigue}\}.$$

For COVID-19, the stored set contains five symptoms. Three are selected, giving $3/5 = 0.60$, or 60%. Influenza also matches fever, cough, and fatigue and receives 60%. Pneumonia matches the same three of its five stored symptoms and also receives 60%. Common Cold matches only cough from four stored symptoms, producing 25%.

Table 6: Example scoring for fever, cough, and fatigue

Condition	Matching evidence	Calculation	Score
Influenza (Flu)	Fever, cough, fatigue	3/5	60%
COVID-19	Fever, cough, fatigue	3/5	60%
Pneumonia	Fever, cough, fatigue	3/5	60%
Chickenpox	Fever, fatigue	2/4	50%
Common Cold	Cough	1/4	25%

When scores tie, Python's list sorting is stable, so tied entries retain their original order in **CASES**. In the example, Influenza precedes COVID-19, and COVID-19 precedes Pneumonia. This ordering is deterministic but should not be interpreted as additional medical evidence.

6.3 Algorithm

Listing 3: Core diagnosis and ranking algorithm

```

1 def diagnose(selected_symptoms):
2     results = []
3
4     for case in CASES:
5         matched = set(selected_symptoms) & set(case["symptoms"])
6
7         if matched:
8             score = len(matched) / len(case["symptoms"])
9             results.append({
10                 "name": case["name"],
11                 "score": score,
12                 "matched": matched,
13                 "advice": case["advice"]
14             })
15
16     results.sort(key=lambda x: x["score"], reverse=True)
17     return results

```

If there are n conditions, each containing at most k symptoms, converting and intersecting the small sets gives an approximate time cost of $O(nk)$, followed by $O(r \log r)$ sorting for r matched results. Memory usage is $O(rk)$ in the worst case because each result stores its matching set. With 24 cases and short symptom lists, both costs are negligible for interactive use.

7 Graphical User Interface

7.1 Window and visual organization

The root window is titled *Medical Expert System*, uses an initial size of 900×700 pixels, and applies a pale blue-gray background. A dark blue header establishes visual hierarchy. The content is divided into symptom selection, action buttons, and an output area. Green is used for the affirmative diagnosis action and red for clearing input, reflecting familiar interface conventions.

7.2 Scrollable symptom checklist

The symptom frame is placed inside a canvas because the number of unique symptoms exceeds the comfortable height of a fixed window. A `<Configure>` event updates the canvas scroll region whenever the embedded frame changes size. Symptoms are displayed in two columns: integer division determines the row, and the remainder after division by two determines the column. Each checkbox is connected to a `BooleanVar` stored in `self.symptom_vars` under its symptom string.

Table 7: GUI widgets and their purposes

Widget	Location or method	Purpose
Frame	Header and button area	Groups related controls and manages layout.
Label	Header	Displays the application title and subtitle.
LabelFrame	Symptoms and results	Gives each major area a visible, descriptive boundary.
Canvas	Symptom area	Provides a scrollable viewport for a long checklist.
<code>ttk.Scrollbar</code>	Symptom area	Moves vertically through symptoms that exceed the viewport.
Checkbutton	Embedded symptom frame	Captures a Boolean choice for each symptom.
Button	Action area	Calls either <code>run_diagnosis()</code> or <code>clear_selection()</code> .
Text	Result area	Presents multi-line, wrapped, read-only output.
messagebox	Input validation	Warns the user when no symptom has been selected.

7.3 Result presentation and widget state

The result widget begins in the disabled state so users cannot accidentally modify the generated explanation. To update it, `run_diagnosis()` temporarily changes its state to normal, removes the previous content, inserts the new report, and disables it again. The displayed output contains four kinds of information: selected symptoms, ranked conditions, matching evidence, and advice. It ends with the educational-use notice.

The matching symptoms are stored as a Python set. Sets do not promise a presentation order, so the order of evidence terms may differ between runs or Python environments. Sorting the set before joining it would make the display alphabetically consistent; this is identified as a small future improvement.

8 Implementation Discussion

8.1 Event-driven execution

The main block creates `tk.Tk()`, passes it to the GUI class, and calls `mainloop()`. The event loop waits for interactions rather than executing the diagnosis immediately. Button `command` arguments store references to handler methods, allowing Tkinter to call the appropriate method when an event occurs.

8.2 Input validation

The program rejects an empty selection before calling the inference engine. The warning is important for usability because an empty selection would otherwise produce a blank list of results without explaining what the user should do. Typed spelling errors cannot occur because input comes from predefined checkboxes.

8.3 Separation of concerns

The knowledge base and `diagnose()` function do not depend on Tkinter. This separation means the reasoning function could be unit tested independently or reused in a command-line or web interface. The GUI is responsible only for gathering state and presenting the function's returned records. For a larger system, the code could be separated into `knowledge_base.py`, `engine.py`, and `gui.py`; the current single-file form is reasonable for a compact educational project.

9 Testing and Evaluation

Testing should verify the inference function separately from the interface and then check their integration. The following cases cover input validation, exact matches, partial matches, ties, output limits, and reset behavior.

Table 8: Proposed functional test cases

ID	Input/action	Expected result	Reason covered
T1	Press <i>Diagnose</i> with no symptoms selected.	A “No Symptoms” warning appears; the result area is unchanged.	Empty-input validation.
T2	Select runny nose, sneezing, sore throat, and cough.	Common Cold receives 100% and appears first.	Exact profile match.
T3	Select fever only.	All cases containing fever receive a partial score; four-symptom cases score 25% and five-symptom cases score 20%.	Denominator and ranking logic.
T4	Select fever, cough, and fatigue.	Influenza, COVID-19, and Pneumonia each receive 60%, in knowledge-base order.	Tie handling and stable sorting.
T5	Select vomiting, diarrhea, and stomach pain.	Gastroenteritis and Food Poisoning each receive 75%.	Overlapping condition profiles.

ID	Input/action	Expected result	Reason covered
T6	Select all visible symptoms and diagnose.	The output contains no more than five conditions, even though many match.	Top-five slicing.
T7	Generate results, then press <i>Clear</i> .	All checkboxes are false and the result area is empty.	Reset behavior.
T8	Scroll to symptoms near the bottom of the list.	Lower entries become accessible and remain selectable.	Scroll-region behavior.
T9	Attempt to type in the result box.	User input is not accepted after generation.	Read-only result state.
T10	Review any completed assessment.	Advice and the non-diagnostic disclaimer are visible.	Explanation and safety communication.

9.1 Illustrative expected outputs

Table 9: Illustrative leading results for selected symptom groups

Selected symptoms	Expected leading condition(s)	Expected score
Runny nose, sneezing, sore throat, cough	Common Cold	100%
Heartburn, chest discomfort, sour taste, belching	Acid Reflux	100%
Red eyes, itchy eyes, watery eyes, eye discharge	Conjunctivitis	100%
Vomiting, diarrhea, stomach pain	Gastroenteritis; Food Poisoning	75% each
Shortness of breath, cough	Asthma; Bronchitis	50% each; Pneumonia 40%
Difficulty sleeping, stress	Insomnia	50%

9.2 Evaluation against objectives

The design satisfies its educational objectives: knowledge is explicit, symptoms are collected through controls, the score is reproducible, and the result includes its matching evidence. It is also maintainable at its current scale. However, success in software behavior must not be confused with clinical accuracy. Passing the test cases confirms that the program implements its rules, not that those rules establish a diagnosis.

10 Limitations, Ethics, and Safety

10.1 Technical and reasoning limitations

Table 10: Principal limitations and their effects

Limitation	Effect	Possible improvement
Small fixed knowledge base	Conditions outside the 24 records can never be suggested.	Use a professionally curated and version-controlled source.
Equal symptom weights	A highly specific symptom and a common symptom contribute equally.	Add validated importance weights and clinical decision rules.
Binary input	Severity, duration, onset, and frequency are ignored.	Collect structured contextual attributes.
Score denominator	Extra selected symptoms do not reduce a condition's score.	Combine coverage with selection precision or another validated similarity measure.
No negative evidence	The absence of a key symptom is not used.	Model required, optional, and exclusionary findings.
No patient context	Age, pregnancy, history, exposures, and medication are absent.	Introduce carefully validated contextual rules with privacy safeguards.
Stable-order ties	Knowledge-base order silently breaks equal scores.	Display tied ranks or add an explicit, justified tie policy.
Unordered matched set	Matching symptoms may be displayed in varying order.	Apply <code>sorted(result["matched"])</code> before joining.
No persistence	Knowledge and results are not saved between sessions.	Add optional, consent-based local storage if a future use case requires it.

The present scoring method can favor a condition with fewer stored symptoms. For example, two matches out of four produce 50%, while the same two matches out of five produce 40%. This behavior follows Equation 2, but it is not proof that the first condition is more likely. A stronger decision-support system would require expert-reviewed rules, validated weights, uncertainty modeling, and careful evaluation on representative clinical data.

10.2 Ethical use

Medical information can influence urgent and consequential decisions. The application should therefore never claim certainty, prescribe medication, discourage professional care, or be deployed as an autonomous triage service. Users may also misread a high match as a probability. The interface

should label the number as a *symptom-profile match* and keep the disclaimer visible near the results rather than relying only on documentation.

The current program has a privacy advantage: it does not save or transmit symptom selections. If later versions add accounts, analytics, cloud services, or history, they would require explicit consent, data minimization, access controls, secure storage, retention rules, and compliance review appropriate to the deployment jurisdiction.

10.3 Emergency considerations

The knowledge base includes potentially serious conditions and breathing-related symptoms, but the engine contains no comprehensive emergency rule layer. A production-oriented health application would need prominently designed escalation guidance for emergency warning signs. Such guidance must be authored and reviewed by qualified healthcare professionals and localized to the user's available emergency services.

11 Future Enhancements

Future work should preserve the transparency of the present design while improving reliability and usability. Appropriate enhancements include:

1. moving the knowledge base to a validated external JSON or database resource with source, reviewer, and revision metadata;
2. adding symptom severity, duration, onset, age group, and relevant exposure questions;
3. separating ordinary matching from an expert-reviewed urgent-warning mechanism;
4. replacing the simple coverage score with a clinically validated method that handles weights, negative evidence, and uncertainty;
5. showing tied results explicitly and sorting displayed matching symptoms;
6. adding automated unit tests for every knowledge-base record and integration tests for GUI handlers;
7. improving accessibility through keyboard navigation checks, scalable fonts, focus indicators, and color-independent status cues;
8. adding a search/filter control if the symptom inventory grows substantially;
9. packaging the code into separate data, engine, interface, and test modules; and
10. obtaining professional review and conducting usability and safety studies before any use beyond classroom demonstration.

12 Conclusion

The Medical Expert System is a clear example of a small rule-based application. Its 24-record knowledge base represents domain facts in ordinary Python data structures; its inference engine uses set intersection and normalized coverage to rank related conditions; and its Tkinter GUI makes the process accessible through checkboxes, buttons, scrolling, and a read-only explanation area. The project also demonstrates good introductory practices such as automatic derivation of unique

symptoms, validation of empty input, separation between reasoning and presentation, and inclusion of a safety notice.

The system's greatest strength is explainability: every displayed result can be traced to specific selected symptoms. Its greatest limitation is that a simple symptom fraction cannot represent clinical diagnosis. The program is therefore successful as an educational expert-system demonstration, provided that its percentages are interpreted only as matches against its small stored profiles and not as medical probabilities or professional conclusions.

A Data Dictionary

Table 11: Principal program data items

Name	Type	Description
CASES	<code>list[dict]</code>	Complete in-memory knowledge base.
<code>case["name"]</code>	<code>str</code>	Human-readable condition label.
<code>case["symptoms"]</code>	<code>list[str]</code>	Lowercase phrases associated with one condition.
<code>case["advice"]</code>	<code>str</code>	General recommendation displayed for that condition.
ALL_SYMPTOMS	<code>list[str]</code>	Sorted, duplicate-free symptom inventory.
selected	<code>list[str]</code>	Symptoms whose GUI Boolean variables are true.
matched	<code>set[str]</code>	Intersection between selected and case symptoms.
score	<code>float</code>	Matched count divided by the case symptom count.
results	<code>list[dict]</code>	Ranked output records created by <code>diagnose()</code> .
symptom_vars	<code>dict[str, BooleanVar]</code>	Mapping from symptom text to checkbox state.

B User Guide

1. Ensure that Python 3 includes Tkinter support, then run the program file.
2. Review the alphabetical symptom checklist and select every applicable item.
3. Use the vertical scrollbar to reach symptoms outside the visible area.
4. Press *Diagnose* to view as many as five profile matches.
5. Read the matching symptoms and general advice for each entry; do not treat the percentage as a diagnosis probability.
6. Press *Clear* to remove all selections and results before starting another example.
7. For actual symptoms or medical concerns, consult an appropriately qualified healthcare professional. Seek urgent help for severe or emergency symptoms.